

LAB PRACTICUM: OpenMP Multi-Core Scaling in Python

Hands-Off Benchmark Practicum • 60 Minutes • Autonomous Execution

Student Name:	Yerassyl Torebek	Student ID:	230103222
Hardware / CPU:	Asus tuf Gaming f15 Intel core i5	Date & Section:	02.10.2026 07-P

📋 LAB PROTOCOL & AUTONOMOUS RULES OF ENGAGEMENT

1. Autonomous Session: You are expected to read, run, optimize, and document all findings independently.
2. Zero Support for Dependency Issues: If you encounter an error, verify that you installed packages inside a clean virtual environment. Compilers are NOT needed for this lab.
3. Equal Hardware Standard: Keep your laptop plugged into wall power and close background browsers, video streams, or games to prevent CPU thermal throttling and frequency capping.
4. Whiteboard Leaderboard: When you finish Task 1 and Task 3, record your CPU model and best speedup factor on the lab whiteboard.

1. Background & Environment Setup (10 Minutes)

Standard CPython executes bytecode under the Global Interpreter Lock (GIL), preventing multiple CPU threads from executing simultaneously in Python space. In this lab, we bypass the GIL using Numba's JIT compiler with its native multithreaded OpenMP/TBB work-stealing engine. Because Numba compiles numeric Python loops directly to machine code and releases the GIL inside parallel loops (prange), we achieve bare-metal native CPU scaling across both macOS (Apple Silicon M-series) and Windows/Linux x86_64 machines with zero local C++ compiler dependencies.

Execute the following in your terminal / command prompt:

```
pip install numpy numba matplotlib
```

2.Challenge 1: The Core Speedometer & Amdahl's Law

Objective: Benchmark an embarrassingly parallel Monte Carlo simulation to approximate π across varying thread counts. You will measure execution latency, quantify parallel speedup, and observe where Amdahl's Law caps performance.

```
import time
import numpy as np
import numba
from numba import njit, prange

print(f"Hardware Threads Detected: {numba.config.NUMBA_NUM_THREADS}")

@njit(parallel=True)
def monte_carlo_pi(n_samples):
    inside_circle = 0
    for i in prange(n_samples):
        x = np.random.uniform(0.0, 1.0)
        y = np.random.uniform(0.0, 1.0)
        if x * x + y * y <= 1.0:
            inside_circle += 1
    return (4.0 * inside_circle) / n_samples

# JIT Warmup (excludes JIT compilation overhead from benchmarks)
_ = monte_carlo_pi(10_000)

SAMPLES = 120_000_000
thread_counts = [1, 2, 4, 8, numba.config.NUMBA_NUM_THREADS]
# Filter out duplicates and invalid counts
thread_counts = sorted(list(set([t for t in thread_counts if t <=
numba.config.NUMBA_NUM_THREADS])))

print(f"{'Threads':<10} | {'Time (s)':<12} | {'Speedup':<10} | {'Efficiency'
('%)':<15}")
print("-" * 55)

t1_baseline = None
for t in thread_counts:
    numba.set_num_threads(t)
    start = time.perf_counter()
    pi_est = monte_carlo_pi(SAMPLES)
    elapsed = time.perf_counter() - start

    if t == 1:
        t1_baseline = elapsed
        speedup = 1.0
        efficiency = 100.0
    else:
```

```

speedup = t1_baseline / elapsed
efficiency = (speedup / t) * 100.0

print(f"{t:<10} | {elapsed:<12.4f} | {speedup:<10.2f}x |
{efficiency:<15.1f}")

```

Deliverables & Technical Questions for Challenge 1:

- A. Record your baseline T_1 (1 thread) and your maximum multi-thread runtime $T_{\max}$ in Table 1 below.

Measurement	Result
Baseline (T_1) — 1 thread	6.9302 seconds
$(T_{\max})$ — 12 threads	1.2266 seconds
Best speedup	5.65×
Efficiency at 12 threads	47.1%

- B. Why is the warm-up call `_ = monte_carlo_pi(10_000)` strictly required before recording benchmark timings?

The first call compiles the function into machine code and initializes the parallel runtime. The warm-up excludes this overhead, so the benchmark measures execution time.

- C. Did your parallel efficiency stay at 100% when scaling from 1 to max threads? What architectural factors (e.g. memory bus contention, hyperthreading/SMT core vs physical core limits) explain the degradation?

No. At 12 threads, efficiency decreased to **47.1%**. Possible reasons include thread management overhead, shared CPU resources, and SMT: logical threads share physical core resources. Memory contention can also limit scaling, although this benchmark does not prove it.

The **128.2% efficiency at 2 threads** shows unusually high speedup. Timing variation or changes in CPU frequency may explain this; repeated measurements are needed to confirm it.

- D. Why does `inside_circle += 1` inside `prange` not corrupt the count? What implicit OpenMP reduction construct does Numba generate here?

Numba recognizes `inside_circle += 1` as a **sum reduction**. Each thread maintains a partial count, and these counts are added together after the loop. This avoids a race condition and is equivalent to the OpenMP construct: `reduction(+:inside_circle)`

3.Challenge 2: Load Imbalance & Dynamic Scheduling

Objective: Compute a high-resolution 2500×2500 Mandelbrot fractal. Unlike Monte Carlo, points in the complex plane require radically different amounts of compute: points inside the set iterate all the way to `max_iter`, while outer points escape in 2-3 iterations. You will investigate how non-uniform workloads impact thread utilization.

```
import time
import numpy as np
import matplotlib.pyplot as plt
from numba import njit, prange

@njit(parallel=True)
def render_mandelbrot_rows(h, w, max_iter):
    img = np.zeros((h, w), dtype=np.int32)
    # Row-parallel loop
    for r in prange(h):
        cy = -1.2 + (r / h) * 2.4
        for c in range(w):
            cx = -2.0 + (c / w) * 2.5
            z_real, z_imag = 0.0, 0.0
            it = 0
            while (z_real * z_real + z_imag * z_imag <= 4.0) and (it <
max_iter):
                next_real = z_real * z_real - z_imag * z_imag + cx
                z_imag = 2.0 * z_real * z_imag + cy
                z_real = next_real
                it += 1
            img[r, c] = it
    return img

@njit(parallel=True)
def render_mandelbrot_cols(h, w, max_iter):
    img = np.zeros((h, w), dtype=np.int32)
    # Column-parallel loop (swapped iteration axis)
    for c in prange(w):
        cx = -2.0 + (c / w) * 2.5
        for r in range(h):
            cy = -1.2 + (r / h) * 2.4
            z_real, z_imag = 0.0, 0.0
            it = 0
            while (z_real * z_real + z_imag * z_imag <= 4.0) and (it <
max_iter):
                next_real = z_real * z_real - z_imag * z_imag + cx
                z_imag = 2.0 * z_real * z_imag + cy
                z_real = next_real
                it += 1
```

```

        img[r, c] = it
    return img

# Warmup JIT
_ = render_mandelbrot_rows(100, 100, 50)
_ = render_mandelbrot_cols(100, 100, 50)

H, W, MAX_IT = 2500, 2500, 1000

t0 = time.perf_counter()
grid_rows = render_mandelbrot_rows(H, W, MAX_IT)
t_rows = time.perf_counter() - t0

t1 = time.perf_counter()
grid_cols = render_mandelbrot_cols(H, W, MAX_IT)
t_cols = time.perf_counter() - t1

print(f"Row-Parallel Render Time:      {t_rows:.3f} s")
print(f"Column-Parallel Render Time: {t_cols:.3f} s")

# Save visual artifact
plt.figure(figsize=(8, 8))
plt.imshow(grid_rows, cmap='magma', extent=[-2.0, 0.5, -1.2, 1.2])
plt.title(f"Mandelbrot {H}x{W} (Render: {t_rows:.2f}s)")
plt.axis('off')
plt.savefig('mandelbrot_output.png', dpi=300, bbox_inches='tight')
print("Saved image: mandelbrot_output.png")

```

Deliverables & Technical Questions for Challenge 2:

- A. Compare `t_rows` vs `t_cols`. Which axis decomposition completed faster on your machine and why?

The row-parallel version took **1.488 seconds**, while the column-parallel version took **1.491 seconds**, using 12 threads. Rows were only **0.003 seconds faster**, so performance was practically the same. This small difference may be normal timing variation; one run is insufficient to establish a clear winner.

- B. Explain how NumPy's C-contiguous (row-major) memory layout interacts with CPU cache lines when threads write to `img[r, c]` vs `img[c, r]`.

NumPy stores neighboring elements of each row next to each other in memory. The row-parallel version writes consecutive elements, making efficient use of cache lines. The column-parallel version writes elements separated by a full row, which can cause more cache misses. Both functions write to `img[r, c]`; their traversal order differs.

- C. Load imbalance analysis: If thread 0 receives the top 20% of rows and thread 2 receives

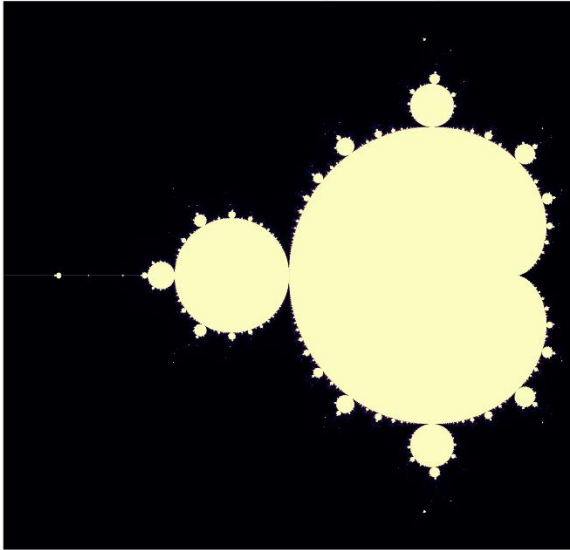
Instructor – sufyan.mustafa@sdu.edu.kz

the center 20% of rows, why does one thread idle while the other struggles? How does OpenMP dynamic scheduling resolve this?

Top rows contain many points that escape after a few iterations, while center rows contain many points that require all 1,000 iterations. A thread processing the top rows finishes early and waits while another thread continues processing the center rows. OpenMP dynamic scheduling assigns additional chunks to available threads, reducing idle time and improving workload balance.

- D. Attach your exported `mandelbrot_output.png` file alongside your completed task sheet.

Mandelbrot 2500x2500 (Render: 1.49s)



4.Challenge 3: Stencil Computation & Memory Bandwidth

Objective: Simulate 2D heat diffusion using a 5-point discrete Laplacian stencil over 300 timesteps on a 1500×1500 grid. Unlike the compute-bound fractal, stencil computations are strictly memory-bandwidth bound. You will determine whether adding CPU cores continues to speed up performance or hits the RAM memory bus wall.

```
import time
import numpy as np
from numba import njit, prange

@njit(parallel=True)
def heat_step(u, u_next, alpha=0.20):
    rows, cols = u.shape
    for i in prange(1, rows - 1):
        for j in range(1, cols - 1):
            u_next[i, j] = u[i, j] + alpha * (
                u[i+1, j] + u[i-1, j] + u[i, j+1] + u[i, j-1] - 4.0 * u[i, j]
            )

GRID_SIZE = 1500
STEPS = 300

u = np.zeros((GRID_SIZE, GRID_SIZE), dtype=np.float64)
u_next = np.zeros_like(u)

# Dirichlet Boundary conditions: Top & Left walls maintained at 100°C
u[0, :] = 100.0
u[:, 0] = 100.0
u_next[0, :] = 100.0
u_next[:, 0] = 100.0

# Warmup
heat_step(u, u_next)

# Benchmark Execution
start = time.perf_counter()
for step in range(STEPS):
    heat_step(u, u_next)
    # Pointer swap (avoids array allocation copies)
    u, u_next = u_next, u
elapsed = time.perf_counter() - start

cells_per_sec = (GRID_SIZE * GRID_SIZE * STEPS) / elapsed / 1e6
print(f"Heat Diffusion Complete: {elapsed:.3f} s")
print(f"Throughput: {cells_per_sec:.2f} Megacells/sec")
```

Deliverables & Technical Questions for Challenge 3:

- A. Record your benchmark runtime and Megacells/sec throughput in Table 1.
- B. Change the array precision from `np.float64` to `np.float32` . Re-run the test. By what factor did runtime decrease? Why does halving memory footprint improve stencil throughput far more than compute capability?
- C. Memory Roofline Model: Explain why adding twice as many CPU cores does not produce a 2x speedup on stencil codes once memory bandwidth is saturated.

5.Student Result Scorecard & Data Collection

Fill in your benchmark results into Table 1 below before submitting this document:

Benchmark Challenge	Active Threads	Array Size / Samples	Time (seconds)	Speedup Factor	Efficiency (%)
Challenge 1: Monte Carlo	1 Thread (Base)	120,000,000	6.9302	1.00x	100.0%
Challenge 1: Monte Carlo	2 Threads	120,000,000	2.7037	2.56x	128.2%
Challenge 1: Monte Carlo	4 Threads	120,000,000	2.9130	2.38x	59.5%
Challenge 1: Monte Carlo	Max Threads	120,000,000	1.2266	5.65x	47.1%
Challenge 2: Mandelbrot (Rows)	Max Threads	2500 x 2500	1.488	N/A	N/A
Challenge 2: Mandelbrot (Cols)	Max Threads	2500 x 2500	1.491	N/A	N/A
Challenge 3: Heat Stencil	Max Threads	1500 x 1500 x 300		N/A	N/A